

Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное
образовательное учреждение высшего образования
«Самарский национальный исследовательский университет имени академика
С.П. Королева»
Институт информатики и кибернетики
Кафедра технической кибернетики

Отчет по лабораторной работе №7

Дисциплина: «ООП»

Тема «Внести изменения в существующий набор типов табулированных функций, позволяющие обрабатывать точки функций по порядку (паттерн «Итератор»), а также выбирать тип объекта табулированной функции при его неявном создании (паттерн «Фабричный метод» и средства рефлексии).»

Выполнил: Зорин Дмитрий
Сергеевич

Группа: 6201-120303D

Самара, 2025

Задание на лабораторную работу

Задание 1

Я расширил интерфейс `TabulatedFunction Iterable<FunctionPoint>` :

(public interface `TabulatedFunction` extends `Function`, `Serializable`, `Cloneable`,
`Iterable<FunctionPoint>`)

А в классах `ArrayTabulatedFunction` и `LinkedListTabulatedFunction` реализован метод `iterator()`, возвращающий анонимный итератор

Пример main:

```
double[] values = {1, 4, 9};
TabulatedFunction arrayFunc =
    new ArrayTabulatedFunction( leftX: 0, rightX: 2, values);

FunctionPoint[] points = {
    new FunctionPoint( x: 0, y: 1),
    new FunctionPoint( x: 1, y: 4),
    new FunctionPoint( x: 2, y: 9)
};

TabulatedFunction listFunc =
    new LinkedListTabulatedFunction(points);

System.out.println("ArrayTabulatedFunction:");
for (FunctionPoint p : arrayFunc) {
    System.out.println(p);
}

System.out.println("LinkedListTabulatedFunction:");
for (FunctionPoint p : listFunc) {
    System.out.println(p);
}
```

Задание 2

Во 2 задании я добавил интерфейс `TabulatedFunctionFactory` :

```
package functions;

public interface TabulatedFunctionFactory { 4 usages 2 implementations
    TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount); 1 usage 2 implementations
    TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values); 1 usage 2 implementations
    TabulatedFunction createTabulatedFunction(FunctionPoint[] points); 1 usage 2 implementations
}
```

С методами для создания объектов

Также описал в классах array и linked классы

фабрик ArrayTabulatedFunctionFactory и LinkedListTabulatedFunctionFactory

```
public static class ArrayTabulatedFunctionFactory implements TabulatedFunctionFactory { no usages
    @Override no usages
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) {
        return new ArrayTabulatedFunction(leftX, rightX, pointsCount);
    }

    @Override no usages
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) {
        return new ArrayTabulatedFunction(leftX, rightX, values);
    }

    @Override no usages
    public TabulatedFunction createTabulatedFunction(FunctionPoint[] points) {
        return new ArrayTabulatedFunction(points);
    }
}
```

Задание 3

В 3 задании добавил в TabulatedFunctions методы

createTabulatedFunction() и использовал конструкторы через рефлексия

Пример в main:

```
TabulatedFunction tf;

tf = TabulatedFunctions.tabulate(cos, leftX: 0, Math.PI, pointsCount: 11);
System.out.println(tf.getClass());

TabulatedFunctions.setTabulatedFunctionFactory(
    new LinkedListTabulatedFunction.LinkedListTabulatedFunctionFactory());
tf = TabulatedFunctions.tabulate(cos, leftX: 0, Math.PI, pointsCount: 11);
System.out.println(tf.getClass());

TabulatedFunctions.setTabulatedFunctionFactory(
    new ArrayTabulatedFunction.ArrayTabulatedFunctionFactory());
tf = TabulatedFunctions.tabulate(cos, leftX: 0, Math.PI, pointsCount: 11);
System.out.println(tf.getClass());

System.out.println("\nПроверка рефлексии");

tf = TabulatedFunctions.createTabulatedFunction(
    ArrayTabulatedFunction.class, leftX: 0, rightX: 10, pointsCount: 3);
System.out.println(tf.getClass());
System.out.println(tf);

tf = TabulatedFunctions.createTabulatedFunction(
    ArrayTabulatedFunction.class, leftX: 0, rightX: 10, new double[]{0, 10});
System.out.println(tf.getClass());
System.out.println(tf);

tf = TabulatedFunctions.createTabulatedFunction(
    LinkedListTabulatedFunction.class,
    new FunctionPoint[]{
        new FunctionPoint(x: 0, y: 0),
        new FunctionPoint(x: 10, y: 10)
    });
```

Пример работы Main:

Проверка интегрирования экспоненты на $[0,1]$

Интеграл $e^x = 1,7182819159$

Шаг = 0,0007812500

Проверка итераторов

ArrayTabulatedFunction:

(0.0; 1.0)

(1.0; 4.0)

(2.0; 9.0)

LinkedListTabulatedFunction:

(0.0; 1.0)

(1.0; 4.0)

(2.0; 9.0)

Проверка фабрик

class functions.ArrayTabulatedFunction

class functions.LinkedListTabulatedFunction

class functions.ArrayTabulatedFunction

Проверка рефлексии

class functions.ArrayTabulatedFunction

{(0.0; 0.0), (5.0; 0.0), (10.0; 0.0)}

class functions.ArrayTabulatedFunction

{(0.0; 0.0), (10.0; 10.0)}

class functions.LinkedListTabulatedFunction

{(0.0; 0.0), (10.0; 10.0)}

class functions.LinkedListTabulatedFunction

{(0.0; 0.0), (0.3141592653589793;
0.3090169943749474), (0.6283185307179586;
0.5877852522924731), (0.9424777960769379;
0.8090169943749475), (1.2566370614359172;
0.9510565162951535), (1.5707963267948966; 1.0),
(1.8849555921538759; 0.9510565162951536),
(2.199114857512855; 0.8090169943749475),
(2.5132741228718345; 0.5877852522924732),

(2.827433388230814; 0.3090169943749475),
(3.141592653589793; 1.2246467991473532E-16)}